Algebraic λ-calculus

Sergey Goncharov
University of Birmingham, UK

Abstract

Abstract

2012 ACM Subject Classification Replace ccsdesc macro with valid one

Keywords and phrases Process algebra, Higer-order GSOS, Strong bisimilarity, CCS

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Acknowledgements I want to thank ...

1 Introduction

{sec:intro}

TODO:
Plan:
- Recall lambda-laws
- introduce $\Sigma \bullet (-)$-laws
- establish a correspondence
- illustrate with examples

2 The Algebraic λ-Calculus

{sec:lambda-algebraic}

We proceed with an approach to modeling the λ-calculus in the framework of HO specifications, and thus enable a congruence result for it as a consequence of **??**. We refer to this presentation as the *algebraic λ-calculus* as it stays in the realm of traditional universal algebra, i.e. algebras for signatures in the category of sets. An alternative, more established way of modeling languages with binders categorically involves presheaf models and binding signatures, and can be accommodated within higher-order abstract GSOS as well, as shown later in **??**.

Recall that the terms of the (untyped) λ-calculus are given by the grammar

$$p, q ::= x \mid \lambda x.p \mid p\, q,$$

with x ranging over a countably infinite set of variables. The standard small-step call-by-name operational semantics is specified by the rules in Figure 1. Here, p, p', q range over λ-terms and $[q/x]$ denotes capture-avoiding substitution of the term q for the variable x. We let $\Lambda(n)$ denote the set of λ-terms (modulo α-equivalence) whose free variables are contained in the set $\{1 \dots, n\}$. In particular, $\Lambda(0)$ is the set of closed λ-terms. Note that the only closed terms that do not reduce under the semantics of Figure 1 are λ-abstractions, which we also conventionally refer to as *values*.

To model the λ-calculus in terms of HO rules, consider the infinite algebraic signature

$$\Sigma = \{\, \mathsf{app}/2 \,\} \cup \{\, [\lambda n + 1.p]/n \mid n \geq 0, p \in \Lambda(n+1) \,\}.$$

As before, app means function application and we write $s\, t$ for $\mathsf{app}(s, t)$. A term of the form $[\lambda n + 1.p](t_1, \dots, t_n)$, where $p \in \Lambda(n+1)$, is supposed to represent the substituted λ-term $(\lambda n + 1.p)[t_1/1, \dots, t_n/n]$.

$$\texttt{app1}\ \frac{p \to p}{p\,q \to p\,q} \qquad \texttt{app2}\ \frac{}{(\lambda x.p)\,q \to p[q/x]}$$

Figure 1 Small-step operational semantics of the call-by-name λ-calculus. {fig:lambda}

$$\frac{}{[\lambda n+1.p](t_1,\dots,t_n) \xrightarrow{t} [p](t_1,\dots,t_n,t)} \quad (p \in \Lambda(n+1))$$

$$\frac{s \to s'}{s\,t \to s'\,t} \qquad \frac{s \xrightarrow{t} s'}{s\,t \to s'}$$

{fig:alg-lambda} **Figure 2** Small-step operational semantics of the algebraic λ-calculus.

{not:lambda-vs-alg-lam

▶ **Notation 2.1.** *We extend the notation $[p](t_1,\dots,t_n)$ from λ-abstractions $p = \lambda n+1.p'$ to arbitrary λ-terms $p \in \Lambda(n)$ by induction as follows:*

- $[i](t_1,\dots,t_n) = t_i$ *if* $i \in \{1,\dots,n\}$;
- $[p\,q](t_1,\dots,t_n) = [p](t_1,\dots,t_n)\ [q](t_1,\dots,t_n)$;
- $[\lambda x.p](t_1,\dots,t_n) = [\lambda n+1.\,p[n+1/x]](t_1,\dots,t_n)$ *for* $x \in \{1,\dots,n+1\}$.

In the case $n = 0$, we write $[p]$ for $[p](t_1,\dots,t_n)$.

The small-step operational semantics of the algebraic λ-calculus is given by the rules of Figure 2, which form an HO specification over Σ modulo renaming variables and completing the premises (cf. **??**). Again, we refer to closed Σ-terms not having unlabeled transitions as *values*; by the above specification, these have the form $[\lambda n+1.p](t_1,\dots,t_n)$.

Our goal is to relate the standard λ-calculus to its algebraic counterpart. At the level of syntax, the corresponding translations are fairly straightforward. Every closed λ-term $p \in \Lambda(0)$ is represented as the constant $[p] \in \mu\Sigma$ in the algebraic λ-calculus. Conversely:

ot:lambda-vs-alg-lambda-2}

▶ **Notation 2.2.** *We define the map $\tau\colon \mu\Sigma \to \Lambda(0)$ by structural induction as follows:*

- $\tau(p\,q) = \tau(p)\,\tau(q)$;
- $\tau([\lambda n+1.p](t_1,\dots,t_n)) = \lambda n+1.p[\tau(t_1)/1,\dots,\tau(t_n)/n]$.

{lem:retr_subst}

▶ **Lemma 2.3.** *Let $p \in \Lambda(n)$ and $t_1,\dots,t_n \in \mu\Sigma$. Then*

$$\tau([p](t_1,\dots,t_n)) = p[\tau(t_1)/1,\ \dots,\ \tau(t_n)/n].$$

In particular ($n = 0$), the map τ is a left inverse of the map $p \mapsto [p]$:

$$\tau([p]) = p \qquad \text{for all } p \in \Lambda(0).$$

Proof. This follows by a straightforward induction on the structure of p. ◀

We show next that the translations of Notation 2.1 and Notation 2.2 are semantically faithful, in the sense that they respect the small-step operational semantics and the ensuing notions of (strong) bisimilarity of terms. On the side of the standard λ-calculus, we work with the following strong version of *applicative bisimilarity* [**?**]:

{def:strong-app}

▶ **Definition 2.4.** Strong applicative bisimilarity *is the greatest relation* $\sim_0^{\mathrm{ap}} \subseteq \Lambda(0) \times \Lambda(0)$ *such that for* $t_1 \sim_0^{\mathrm{ap}} t_2$ *the following conditions hold:*

$$t_1 \to t_1 \implies \exists t_2.\ t_2 \to t_2 \wedge t_1 \sim_0^{\mathrm{ap}} t_2; \tag{A1}$$

$$t_1 = \lambda x.t_1' \implies \exists t_2'.\ t_2 = \lambda x.t_2' \wedge \forall e \in \Lambda(0).\ t_1'[e/x] \sim_0^{\mathrm{ap}} t_2'[e/x]; \tag{A2}$$

$$t_2 \to t_2 \implies \exists t_1.\ t_1 \to t_1 \wedge t_1 \sim_0^{\mathrm{ap}} t_2; \tag{A3}$$

$$t_2 = \lambda x.t_2' \implies \exists t_1'.\ t_1 = \lambda x.t_1' \wedge \forall e \in \Lambda(0).\ t_1'[e/x] \sim_0^{\mathrm{ap}} t_2'[e/x]. \tag{A4}$$

▶ Remark 2.5. **1.** Strong applicative bisimilarity corresponds to coalgebraic bisimilarity on the deterministic LTS with states $\Lambda(0)$, label set $L = \Lambda(0) + \{\_\}$ (where '_' denotes the lack of a label), unlabeled transitions $t \to t'$ as specified in Figure 1, and labeled transitions

$$\lambda x.t \xrightarrow{e} t[e/x] \qquad \text{for all } e \in \Lambda(0).$$

2. Abramsky's original notion of *applicative bisimilarity* is the weak version of Definition 2.4 where unlabeled transitions are treated as unobservable computation steps. It can also be studied in our framework with some additional effort; see Section 3.

{lem:retr_step}

▶ **Lemma 2.6.** *For all* $r, s, t \in \mu\Sigma$*, the following holds:*

{item:retr-step-labeled}

1. *If* $r \xrightarrow{t} s$ *then* $\tau(r) \xrightarrow{\tau(t)} \tau(s)$.

{item:retr-step-unlabeled}

2. *If* $r \to s$ *then* $\tau(r) \to \tau(s)$.

In particular, r *is a value iff* $\tau(r)$ *is a value.*

Proof. **1.** Let $r \xrightarrow{t} s$. Then there exists $p \in \Lambda(n+1)$ and $t_1, \ldots, t_n \in \mu\Sigma$ such that

$$r = [\lambda n+1.\, p](t_1, \ldots, t_n) \qquad \text{and} \qquad s = [p](t_1, \ldots, t_n, t).$$

Therefore, by Lemma 2.3,

$$\tau(r) = \lambda n+1.\, p[\tau(t_1)/1, \ldots, \tau(t_n)/n]$$
$$\xrightarrow{\tau(t)} p[\tau(t_1)/1, \ldots, \tau(t_n)/n, \tau(t)/n+1] = \tau([p](t_1, \ldots, t_n, t)) = \tau(s).$$

2. We proceed by structural induction on r. Suppose that $r \to s$; then $r = u\,v$ for some $u, v \in \mu\Sigma$. There are two cases:

- u is not a value, i.e. $u \to u'$. Then $r = u\,v \to u'\,v = s$, and moreover $\tau(u) \to \tau(u')$ induction, which implies

$$\tau(t) = \tau(u)\,\tau(v) \to \tau(u')\,\tau(v) = \tau(s).$$

- u is a value, i.e. $u = [\lambda n+1.\, p](t_1, \ldots, t_n)$ for some $p \in \Lambda(n+1)$ and $t_1, \ldots, t_n$. Then $u \xrightarrow{v} [p](t_1, \ldots, t_n, v)$ and therefore

$$r = u\,v \to [p](t_1, \ldots, t_n, v) = s.$$

It follows by Lemma 2.3 that

$$\tau(r) = \tau([\lambda n+1.\, p](t_1, \ldots, t_n)\,v) = \lambda n+1.\, p[\tau(t_1)/1, \ldots, \tau(t_n)/n]\,\tau(v)$$
$$\to p[\tau(t_1)/1, \ldots, \tau(t_n)/n, \tau(v)/n+1] = \tau([p](t_1, \ldots, t_n, v)) = \tau(s). \qquad \blacksquare$$

◀

Let $\sim$ denote the coalgebraic bisimilarity relation on the LTS on $\mu\Sigma$ induced by the rules of the algebraic λ-calculus (Figure 2). It relates to applicative bisimilarity as follows:

{pro:cl-lambda}

▶ **Proposition 2.7.** **1.** *For all $s,t \in \mu\Sigma$, $s \sim t$ iff $\tau(s) \sim_0^{\mathrm{ap}} \tau(t)$.*
2. *For all $p,q \in \Lambda(0)$, $p \sim_0^{\mathrm{ap}} q$ iff $[p] \sim [q]$.*

Proof. **1.** ($\Leftarrow$) Define the relation $R \subseteq \mu\Sigma \times \mu\Sigma$ by

$$s\,R\,t \qquad \text{iff} \qquad \tau(s) \sim_0^{\mathrm{ap}} \tau(t).$$

It suffices to show that R is a bisimulation, which implies $R \subseteq \sim$ and hence proves the claim. Thus suppose that $s\,R\,t$ and $s \xrightarrow{r} s'$. By Lemma 2.61 we have $\tau(s) \xrightarrow{\tau(r)} \tau(s')$. Since $\tau(s) \sim_0^{\mathrm{ap}} \tau(t)$ we know that $\tau(t)$ has a $\tau(r)$-labeled transition, which by Lemma 2.61 must be of the form $\tau(t) \xrightarrow{\tau(r)} \tau(t')$ where $t \to t'$, and moreover $\tau(s') \sim_0^{\mathrm{ap}} \tau(t')$; hence $s'\,R\,t'$. The case of unlabeled transitions is analogous, using Lemma 2.62.

($\Rightarrow$) Define the relation $R \subseteq \Lambda(0) \times \Lambda(0)$ by

$$R = \{\, (\tau(s),\tau(t)) \colon s,t \in \mu\Sigma,\ s \sim t \,\}.$$

It suffices to show that R is an applicative bisimulation, which implies $R \subseteq \sim_0^{\mathrm{ap}}$ and hence proves the claim. Thus suppose that $s \sim t$, so $\tau(s)\,R\,\tau(t)$. If $\tau(s)$ has an unlabeled transition, then $\tau(s) \to \tau(s')$ where $s \to s'$ by Lemma 2.62. Since $s \sim t$, it follows that $t \to t'$ and $t \sim t'$; hence $\tau(t) \to \tau(t')$ and $\tau(s')\,R\,\tau(t')$. If $\tau(s)$ is a λ-abstraction, then

$$s = [\lambda n+1.p](t_1,\dots,t_n) \qquad \text{and} \qquad \tau(s) = \lambda n+1.p[\tau(t_1)/1,\dots,\tau(t_n)/n]$$

for some $p \in \Lambda(n+1)$ and $t_1,\dots,t_n \in \mu\Sigma$ by definition of τ. For every $e \in \Lambda(0)$ we have $s \xrightarrow{[e]} [p](t_1,\dots,t_n,[e])$. Since $s \sim t$, it follows that $t = [\lambda m+1.q](t'_1,\dots,t'_m)$ for some $q \in \Lambda(m+1)$ and $t'_1,\dots,t'_m \in \mu\Sigma$, and $t \xrightarrow{[e]} [q](t'_1,\dots,t'_m,[e])$, hence

$$[p](t_1,\dots,t_n,[e]) \sim [q](t'_1,\dots,t'_m,[e])$$

because $\sim$ is a bisimulation. Therefore, by Lemma 2.3 and Lemma 2.6, we get

$$\tau(s) \xrightarrow{e} p[\tau(t_1)/1,\dots,\tau(t_n)/n,e/n+1] = \tau([p](t_1,\dots,t_n,[e]))$$

and

$$\tau(t) \xrightarrow{e} q[\tau(t'_1)/1,\dots,\tau(t'_m)/n,e/m+1] = \tau([q](t'_1,\dots,t'_m,[e])),$$

and $\tau([p](t_1,\dots,t_n,[e]))\,R\,\tau([q](t'_1,\dots,t'_m,[e]))$, as required.

2. is an immediate consequence of (1) and Lemma 2.3: For $p,q \in \Lambda(0)$, we have $[p] \sim [q]$ iff $p = \tau([p]) \sim_0^{\mathrm{ap}} \tau([q]) = q$. ■

◀

▶ Remark 2.8. The above proposition allows us to establish congruence properties for the λ-calculus by reducing them to the algebraic λ-calculus. For instance, let us show that $p \sim_0^{\mathrm{ap}} q$ implies $r\,p \sim_0^{\mathrm{ap}} r\,q$ for all $p,q,r \in \Lambda(0)$. Indeed, $p \sim_0^{\mathrm{ap}} q$ entails $[p] \sim [q]$ by Proposition 2.7(2). Therefore, by **??**, we have

$$[r\,p] = [r]\,[p] \sim [r]\,[q] = [r\,q],$$

whence $r\,p \sim_0^{\mathrm{ap}} r\,q$ by Proposition 2.7(2) again. A congruence result that directly applies to the λ-calculus will be derived in **??**.

2.1 λ-Laws

We begin by equipping our ambient category $\mathbf{C}$ with a closed monoidal structure $(\mathbf{C}, V, \bullet, \multimap)$, meant to abstract from the internal mechanisms of variable management [1]. Monoidal closedness yields a natural isomorphism $(\text{–})^\flat \colon \mathbf{C}(X \bullet Y, Z) \to \mathbf{C}(X, Y \multimap Z)$, which we will need below.

▶ **Definition 2.9** (Pointed Strength [1, 3]). *Let us denote by j the forgetful functor from $V/\mathbf{C}$ to $\mathbf{C}$, sending a morphism $V \to X$ to X. Objects of $V/\mathbf{C}$ are called* (V-)pointed objects *of $\mathbf{C}$.*

A (V-)pointed strength *on an endofunctor $F \colon \mathbf{C} \to \mathbf{C}$ is a family of morphisms $\mathrm{st}_{X,Y} \colon FX \bullet jY \to F(X \bullet jY)$, natural in $X \in \mathbf{C}$ and $Y \in V/\mathbf{C}$, such that the following diagrams commute:*

$$\begin{array}{ccc} & FX & \\ \swarrow & & \searrow \\ FX \bullet V & \xrightarrow{\mathrm{st}_{X,V}} & F(X \bullet V) \end{array}$$

$$\begin{array}{ccccc} (FX \bullet jY) \bullet jZ & \xrightarrow{\mathrm{st}_{X,Y} \bullet jZ} & F(X \bullet jY) \bullet jZ & \xrightarrow{\mathrm{st}_{X \bullet jY, Z}} & F((X \bullet jY) \bullet jZ) \\ \| & & & & \| \\ FX \bullet (jY \bullet jZ) & & \xrightarrow{\mathrm{st}_{X, Y \bullet Z}} & & F(X \bullet (jY \bullet jZ)) \end{array}$$

(eliding the names of the canonical isomorphisms).

We then assume that Σ has the form $V + \Sigma'$ and that the functor Σ is V-strong. This guarantees that the initial algebra $\mu\Sigma$ (if it exists) is a monoid [2, Theorem 4.1], whose multiplication we denote as $\mathsf{subst} \colon \mu\Sigma \bullet \mu\Sigma \to \mu\Sigma$.

▶ **Definition 2.10** (λ-law). *Let $B \colon \mathbf{C}^{\mathrm{op}} \times \mathbf{C} \to C$ be a bifunctor in $\mathbf{C}$, $\Theta \colon \mathbf{C} \to \mathbf{C}$ be a V-pointed strong endofunctor and let $\Xi \colon \mathbf{C} \to \mathbf{C}$ be an endofunctor of the form $V + \Xi'$ where Ξ' is V-pointed strong. A* λ-law of (Ξ, Θ) over B *consists of two ingredients:*

$$\xi_{X,Y} \colon \Xi(X \times B(X,Y)) \to B(X, \Xi^\star(X + Y))$$
$$\theta_{X,Y} \colon \Theta(jX \multimap Y) \to jX \multimap B(jX, Y).$$

The first one is a (0-pointed) higher-order GSOS law; the second one is a family of morphisms dinatural in $X \in V/\mathbf{C}$ and natural in $Y \in \mathbf{C}$, such that the following pentagon commutes:

$$\begin{array}{ccccc} \Theta(X \multimap Y) \bullet (X \multimap X) \bullet X & & \xrightarrow{\mathrm{st}^{\Theta}_{X \multimap Y, X \multimap X} \bullet X} & & \Theta((X \multimap Y) \bullet (X \multimap X)) \bullet X \\ {\scriptstyle \Theta(X \multimap Y) \bullet \mathrm{ev}_{X,X}} \downarrow & & & & \downarrow {\scriptstyle \Theta \mathrm{comp}_{X,X,Y} \bullet X} \\ \Theta(X \multimap Y) \bullet X & & & & \Theta(X \multimap Y) \bullet X \\ & {\scriptstyle \theta^\sharp_{X,Y}} \searrow & & \swarrow {\scriptstyle \theta^\sharp_{X,Y}} & \\ & & B(X,Y) & & \end{array} \tag{1}$$

(eliding the obvious associativity transformation). Note the the use of strength here is legit, since $X \multimap X$ is canonically pointed.

In a λ-law, functors Ξ and Θ model the syntax of the language and ξ and θ the semantics. The 0-pointed higher-order GSOS law ξ follows the classic (higher-order) GSOS paradigm. As for θ, it abstractly captures the fact that the behaviour of a λ-abstraction is determined by

the internal substitution structure of its body in a well-behaved manner. Concretely, it allows one to show that if the body of a λ-abstraction satisfies the logical predicate, then so does the λ-abstraction itself. The λ-laws can be understood as a proper subset of higher-order GSOS laws for languages with variable binding. In particular, any λ-law $(\Xi, \Theta, B, \xi, \theta)$ determines a V-pointed higher-order GSOS law

{{eq:lamtogsos}}

$$\rho_{X,Y} \colon \Sigma(X \times (X \multimap Y) \times B(X,Y)) \to (X \multimap \Sigma^\star(X+Y)) \times B(X, \Sigma^\star(X+Y)) \tag{2}$$

of $\Sigma = \Xi + \Theta$ over $(- \multimap -) \times B$, and therefore the previously developed theory of abstract higher order GSOS laws can (and will) be reused. For any $X \in V/\mathbf{C}$, and $Y \in \mathbf{C}$ we define the trivial substitution on the substitution structure $X \multimap Y$:

$$\mathrm{triv}_{X,Y} \colon X \multimap Y \cong (X \multimap Y) \bullet V \xrightarrow{(\mathrm{var}_X \multimap Y)^\sharp} Y.$$

Then we define the components of (2) as copairs $[\langle \rho_{11}^\flat, \rho_{12} \rangle, \langle \rho_{21}^\flat, \rho_{22} \rangle]$ where

$$\begin{aligned}
\rho_{11} \colon \Xi(X \times (X \multimap Y) \times B(X,Y)) \bullet X &\xrightarrow{\Xi(\mathsf{fst}\cdot\mathsf{snd})\bullet X} \Xi(X \multimap Y) \bullet X \cong X + \Xi'(X \multimap Y) \bullet X \\
&\xrightarrow{X + \mathrm{st}^{\Xi'}_{X \multimap Y, X}} X + \Xi'((X \multimap Y) \bullet X) \\
&\xrightarrow{X + \Xi' \mathrm{ev}_{X,Y}} X + \Xi' Y \xrightarrow{[\eta\cdot\mathsf{inl}, \Sigma^\star\,\mathsf{inr}]} \Sigma^\star(X+Y), \\
\rho_{12} \colon \Xi(X \times (X \multimap Y) \times B(X,Y)) &\xrightarrow{\Xi(\mathsf{id}\times\mathsf{snd})} \Xi(X \times B(X,Y)) \xrightarrow{\xi_{X,Y}} B(X, \Xi^\star(X+Y)) \\
&\xrightarrow{B(X, \mathsf{inl}^\sharp)} B(X, \Sigma^\star(X+Y)), \\
\rho_{21} \colon \Theta(X \times (X \multimap Y) \times B(X,Y)) \bullet X &\xrightarrow{\Theta(\mathsf{fst}\cdot\mathsf{snd})\bullet X} \Theta(X \multimap Y) \bullet X \xrightarrow{\mathrm{st}^{\Theta}_{X \multimap Y, X}} \Theta((X \multimap Y) \bullet X) \\
&\xrightarrow{\Theta \mathrm{ev}_{X,Y}} \Theta Y \xrightarrow{\mathsf{inr}^\sharp} \Sigma^\star Y \xrightarrow{\Sigma^\star\,\mathsf{inr}} \Sigma^\star(X+Y), \\
\rho_{22} \colon \Theta(X \times (X \multimap Y) \times B(X,Y)) &\xrightarrow{\Theta(\mathsf{fst}\cdot\mathsf{snd})} \Theta(X \multimap Y) \xrightarrow{\theta_{X,Y}} X \multimap B(X,Y) \\
&\xrightarrow{\mathrm{triv}_{B(X,Y)}} B(X,Y) \xrightarrow{B(X, \eta\cdot\mathsf{inr})} B(X, \Sigma^\star(X+Y)).
\end{aligned}$$

The family ρ is dinatural in X because all of its components are. We consider the canonical operational model of a λ-law to be the canonical model of the derived higher-order GSOS law, which in this case is a pair of morphisms

$$\langle \phi, \gamma \rangle \colon \mu\Sigma \to (\mu\Sigma \multimap \mu\Sigma) \times B(\mu\Sigma, \mu\Sigma).$$

Map $\gamma \colon \mu\Sigma \to B(\mu\Sigma, \mu\Sigma)$ models the computational behaviour terms. In addition, the initial $(\Xi + \Theta)$-algebra $\mu\Sigma$ is a $\bullet$-monoid with $\phi \colon \mu\Sigma \to (\mu\Sigma \multimap \mu\Sigma)$ as the multiplication and the inclusion of variables as unit. In fact, $\mu\Sigma$ is a $\Xi' + \Theta$-monoid [**?** , §4], a fact we prove in the next lemma.

▶ **Notation 2.11.** *In the sequel, we use* $\iota^{\Xi'} = \iota \cdot \mathsf{inr} \cdot \mathsf{inl} \colon \Xi' \mu\Sigma \to \mu\Sigma$, $\iota^{\Xi} = \iota \cdot \mathsf{inl} \colon \Xi\mu\Sigma \to \mu\Sigma$, *and* $\iota^{\Theta} = \iota \cdot \mathsf{inr} \colon \Theta\mu\Sigma \to \mu\Sigma$. *The canonical point of* $\mathrm{var}_{\mu\Sigma} \colon V \to \mu\Sigma$ *is* $\mathrm{var}_{\mu\Sigma} = \iota^V = \iota \cdot \mathsf{inl} \cdot \mathsf{inl}$. *In addition, as* $\mu\Sigma = (V + \Xi' + \Theta)^\star(0) = (\Xi' + \Theta)^\star(V)$, $\mathrm{var}_{\mu\Sigma} = \eta_V$.

189 # 3 Conclusions and Further Work

{sec:concl}

190 ## References

191 **1** Marcelo P. Fiore. Second-order and dependently-sorted abstract syntax. In *23d Annual*
192 *IEEE Symposium on Logic in Computer Science (LICS 2008)*, pages 57–68. IEEE Computer
193 Society, 2008. doi:10.1109/LICS.2008.38.

194 **2** Marcelo P. Fiore, Gordon D. Plotkin, and Daniele Turi. Abstract syntax and variable
195 binding. In *14th Annual IEEE Symposium on Logic in Computer Science (LICS 1999)*,
196 pages 193–202. IEEE Computer Society, 1999. doi:10.1109/LICS.1999.782615.

197 **3** André Hirschowitz, Tom Hirschowitz, Ambroise Lafont, and Marco Maggesi. Variable
198 binding and substitution for (nameless) dummies. In *25th International Conference on*
199 *Foundations of Software Science and Computation Structures (FOSSACS 2022)*, volume
200 13242 of *LNCS*, pages 389–408. Springer, 2022. doi:/10.1007/978-3-030-99253-8_20.

A Omitted Proofs

A.1 Proof of ??